

4. Паттерны надёжности

Надёжность — ключевой фактор для соответствия SLA 99.9% и соблюдения жёстких требований DSP по latency. В архитектуре AdScale применяются следующие паттерны и стратегии.

4.1 Circuit Breaker для синхронных вызовов сервисов

Применяется на всех синхронных зависимостях.

Возможные состояния и пороги срабатывания:

- Closed -нормальная работа, **позволяет** запросам проходить к защищаемому сервису;
- Open — Circuit Breaker **блокирует** все попытки выполнить запрос к защищаемому сервису. **Порог срабатывания:**
 - Серия 10 ошибок подряд;
 - Более 90 % ошибок за период времени (30 сек).При срабатывании Сервис перестаёт делать запросы и сразу возвращает заранее заготовленный ответ (дефолт/кэш).
Переход в Half-Open - через 15 сек.
- Half-Open — переходное состояние, в котором Circuit Breaker начинает частично (10 процентов от нагрузки или 10 запросов подряд) разрешать запросы к сервису для тестирования его доступности и надёжности.
 - Отсутствие ошибок — переход в состояние в Closed
 - В случае повторения ошибок согласно критериям порога Open — переход в состояние в Open.

4.2 Паттерн Retry для синхронных вызовов сервисов

Используется паттерн Retry (Повторные попытки) с экспоненциальной задержкой и джиттером. Применяется в случае возникновения временных (сетевых) ошибок, таймаутов, 503 и пр.

Пороги Retry для сервиса ставок (низкая задержка):

- максимум 2–3 попытки;
- экспоненциальная задержка: 50 мс → 100 мс → 200 мс;
- джиттер (random) $\pm 20\%$ к задержке, чтобы избежать «шторма»;
- общий бюджет времени на все повторы : $\leq 20\text{--}30$ мс, чтобы не превысить SLA 80–100 мс.

Указанные значения порогов являются ориентировочными и должны быть проверены при функциональном и нагрузочном тестировании, а также могут быть скорректированы в процессе эксплуатации системы.

4.2 Идемпотентность финансовых операций

Финансы нельзя «начислить дважды» из-за повторов или дублей сообщений.

Для реализации транзакционности в финансовых операциях используется паттерн SAGA. Каждое финансовое событие (например, списание за показ) содержит уникальный идентификатор `transaction_id` (генерируется на основе `auction_id + impression_id`).

- В таблице транзакций создаётся уникальное ограничение на `transaction_id`.
- Перед выполнением списания сервис проверяет наличие записи с таким ID в БД:
- Если запись существует - игнорирует (идемпотентный ответ).
- Если нет - выполняет списание и вставляет запись в той же транзакции.

Для дополнительной защиты, если БД временно недоступна, сервис может временно сохранять обработанные ID в Redis-сети (с TTL, например, 24 часа) для быстрой проверки.

4.3 Резервные стратегии (Fallback / Graceful Degradation)

При недоступности зависимых сервисов или превышении тайм-аутов, система должна продолжать работать, пусть и с пониженной точностью.

Дефолтные ставки (Fallback)

Если Redis/PostgreSQL недоступны, Bidding Service может:

- вернуть «нет рекламы» (safest);
- отдать дефолтную ставку по паре `ad_unit + geo` из локального конфига или из Redis с долгим TTL;
- применить упрощённые правила: без таргетинга по устройствам/категориям, только гео и статус кампании.

Стратегия деградации функционала (Degradation)

- Полная потеря Redis: использовать только PostgreSQL replica (медленнее, но надёжно).
- Потеря PostgreSQL replica: работать только по кэшу или с мастер данными.
- Потеря Kafka: события можно временно не писать (или писать в локальный буфер/лог), но ставки продолжать отдавать.

4.4 Надежность инфраструктуры

С целью соблюдения требуемого SLA 99,9 необходимо повысить надежность ключевых инфраструктурных сервисов:

- **Redis** - используется в отказоустойчивой конфигурации с помощью инструмента Sentinel. Redis Sentinel следит за одним мастером и переключает его на реплику при сбое, сохраняя единый узел для записи. Redis Sentinel обладает следующими преимуществами: Простая настройка и архитектура кластера (по сравнению с Redis Cluster) и дальнейшая поддержка. Оба варианта могут быть реализованы как в случае on premise так и в случае выбора облачной инфраструктуры с нужными параметрами отказоустойчивости .
- **Kafka** – в кластерной конфигурации Zookeeper /Kraft . Кластеризация Apache Kafka с помощью ZooKeeper — это объединение нескольких серверов Kafka (брокеров) в единую надежную систему, где ZooKeeper работает как «дирижер» или «смотритель зоопарка», координируя их работу, храня общие настройки и следя за состоянием каждого брокера Kafka. Возможна гео распределённая кластеризация с репликацией данных между всеми брокерами Kafka
- **PostgreSQL** – в кластерной конфигурации высокой доступности Patroni. Для каждой БД используется два сервера для хранения данных и etcd сервер для хранения состояния кластера. Для задач AD Scale необходимо использовать кластер с синхронной репликацией, что повышает требование к сети и сетевой задержки.

Кластеризация возможна:

- В рамках одного ЦОД или зоны доступности облачного провайдера. Подходит в качестве стартовой конфигурации
- Использование двух ЦОД/ зон доступности и использование геораспределённой конфигурации кластеров для повышения надежности. Использование возможно, если сетевая задержка между используемыми ЦОД не превышает 5-10 мс.
- Использование нескольких географически распределённых ЦОД/ зон доступности, реализация в рамках ЦОД - отказоустойчивого кластера.